

AutoCiclo

Documentación Técnica del Proyecto

Yalil Musa Talhaoui · CFGS DAM 2º · IES P. Hermenegildo Lanz, Granada · 2025/2026

AutoCiclo — Documentación Técnica del Proyecto

Autor: Yalil Musa Talhaoui
Centro: IES Pedro Hermenegildo Lanz, Granada
Ciclo: CFGS Desarrollo de Aplicaciones Multiplataforma (DAM) — 2º Curso
Curso: 2025/2026
Repositorio: <https://github.com/yalilms/autociclo-tfg>

1. Resumen del Proyecto

AutoCiclo es un ecosistema multiplataforma de gestión integral para empresas de desguace de vehículos. El sistema permite controlar el ciclo de vida completo de un vehículo desde su entrada al desguace hasta la venta de sus piezas a clientes.

El proyecto consta de **cuatro aplicaciones interconectadas** que trabajan sobre la misma API REST y base de datos:

Aplicación	Plataforma	Destinatarios
Autociclo Desktop	Java 21 + JavaFX	Administradores del desguace
Autociclo Shop	Web (React + Vite)	Clientes que buscan y solicitan piezas
Autociclo Worker	Móvil (React Native + Expo)	Empleados del almacén
API REST	Spring Boot 3	Backend común a todas las apps

¿Qué resuelve?

Un desguace gestiona cientos de vehículos y miles de piezas. Sin un sistema digitalizado:

- El stock se desconoce en tiempo real
- Los clientes llaman por teléfono para preguntar disponibilidad
- Los empleados buscan piezas a mano sin saber dónde están
- La facturación se hace en papel o en hojas de cálculo

AutoCiclo digitaliza todo este proceso: los empleados escanean piezas con el móvil, los clientes solicitan presupuestos desde la web, los administradores aprueban solicitudes desde el escritorio y el sistema genera automáticamente el pedido de venta en Odoo.

2. Tecnologías Utilizadas

Backend

Tecnología	Versión	Uso
Java	21 LTS	Lenguaje del backend y desktop
Spring Boot	3.2.x	Framework API REST
Spring Security	6.x	Autenticación y autorización
JWT (jjwt)	0.12.x	Tokens de acceso stateless
Spring Data JPA	3.2.x	ORM sobre Hibernate
Spring AMQP	3.x	Integración con RabbitMQ
Maven	3.9.x	Gestión de dependencias del backend

Base de Datos

Tecnología	Versión	Uso
MySQL	8.0	Base de datos relacional principal
HikariCP	5.x	Pool de conexiones

Mensajería

Tecnología	Versión	Uso
RabbitMQ	3.13.7	Broker de mensajes asíncronos
Docker	24.x	Contenedor para RabbitMQ en el servidor

ERP / Facturación

Tecnología	Versión	Uso
Odoo Community	17	CRM, pedidos de venta y facturación
JSON-RPC	—	Protocolo de integración con Odoo

Aplicación Web

Tecnología	Versión	Uso
React	19	Librería de interfaz de usuario
Vite	5.x	Bundler y servidor de desarrollo
TypeScript	5.x	Tipado estático
Tailwind CSS	3.x	Estilos utilitarios
React Router	v6	Enrutamiento SPA
Axios	1.x	Cliente HTTP
Zustand	4.x	Gestión de estado global
Nginx	1.x	Servidor web para el dist de producción

Aplicación Desktop

Tecnología	Versión	Uso
JavaFX	21	Framework de interfaz gráfica
Gradle	8.x	Gestión de dependencias
OkHttp	4.x	Cliente HTTP para la API
Jackson	2.x	Deserialización JSON

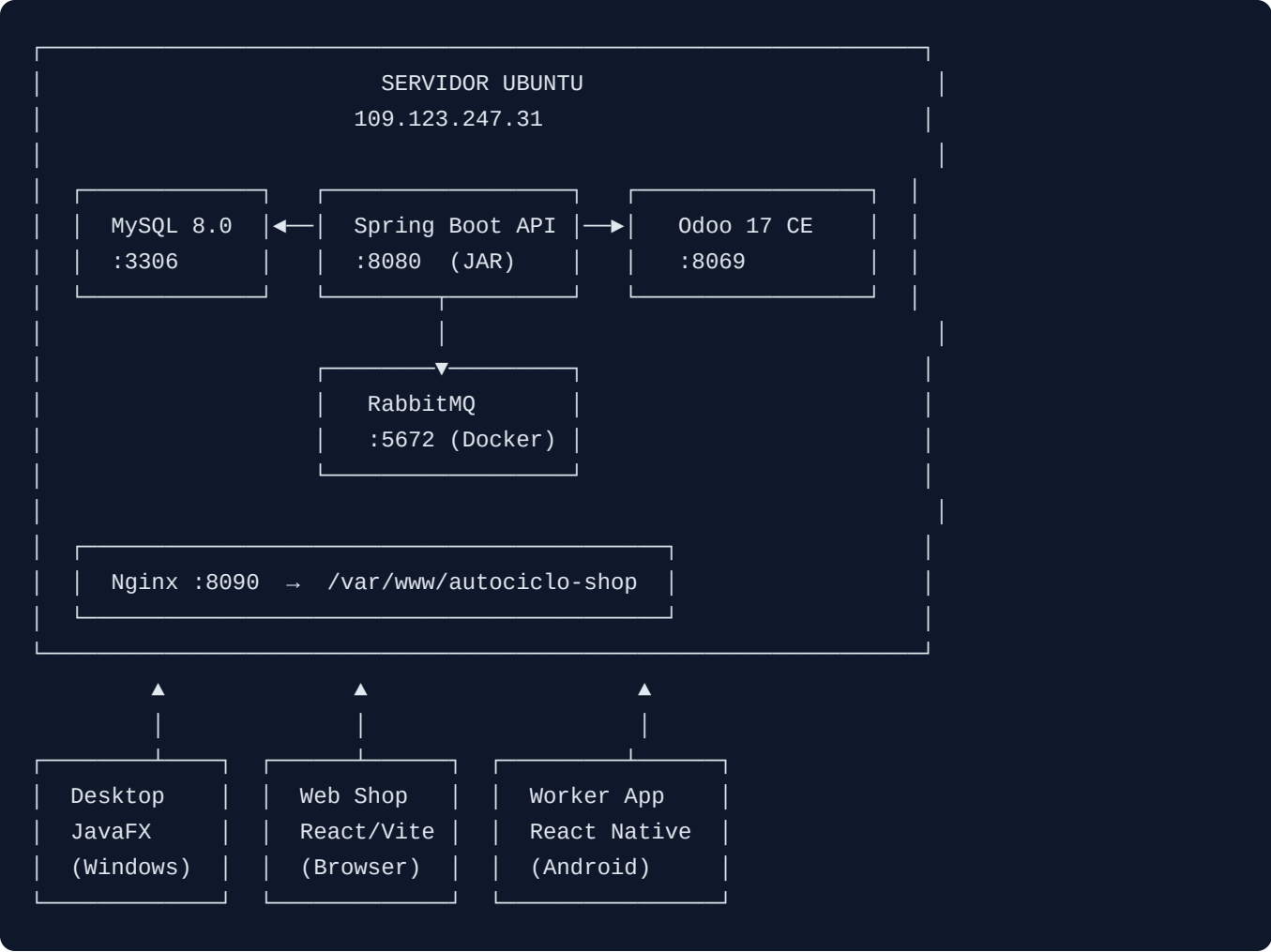
Aplicación Móvil

Tecnología	Versión	Uso
React Native	0.81	Framework móvil multiplataforma
Expo SDK	54	Toolchain y acceso a APIs nativas
Expo Router	6.x	Enrutamiento basado en ficheros
NativeWind	4.x	Tailwind CSS para React Native
expo-camera	—	Escaneo de códigos QR
expo-secure-store	—	Almacenamiento cifrado del JWT
Zustand	4.x	Gestión de estado global
Axios	1.x	Cliente HTTP

Infraestructura

Componente	Detalle
Servidor	VPS Ubuntu 22.04 LTS — Contabo (4 vCPU, 8 GB RAM, 256 GB SSD)
IP pública	109.123.247.31
API	Puerto 8080 — Spring Boot JAR con systemd
Web Shop	Puerto 8090 — Nginx sirviendo el dist de React
Odoo	Puerto 8069 — proceso Python directo
RabbitMQ	Puertos 5672/15672 — contenedor Docker
MySQL	Puerto 3306 — proceso local (solo acceso interno)

3. Arquitectura del Sistema



Flujo principal de mensajería (RabbitMQ)

```

Cliente (Web) → POST /api/solicitudes
              |
              ▼
Spring Boot publica en
cola [solicitudes.nueva]
              |
              ▼
Desktop recibe alerta
en tiempo real (badge)
  
```

```

Empleado (Worker) → POST /api/stock/movimiento
                  | (si stock < mínimo)
                  ▼
Spring Boot publica en
cola [stock.alerta]
                  |
                  ▼
Worker Dashboard actualiza
(polling 30s)
  
```

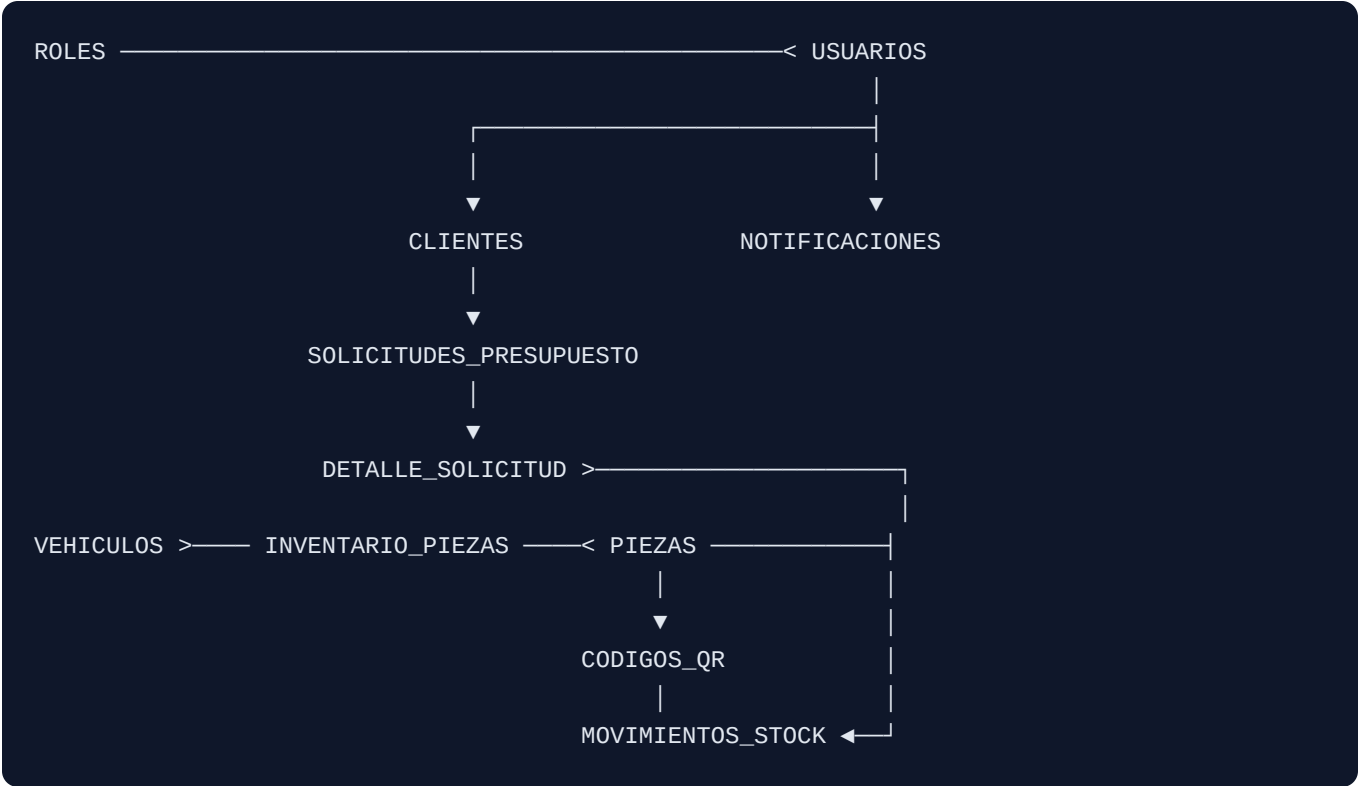
Flujo de aprobación y facturación (Odoo)

```

Admin (Desktop)
└─> PUT /api/solicitudes/{id}/aprobar
    |
    ▼
Spring Boot llama Odoo JSON-RPC
├─> Busca/crea cliente en Odoo
├─> Crea Pedido de Venta
└─> Devuelve referencia (ej: S0/2026/0001)
    |
    ▼
Odoo genera factura PDF automáticamente
Cliente ve referencia Odoo en "Mis Solicitudes"
  
```

4. Modelo Entidad-Relación

Diagrama (notación simplificada)



Cardinalidades clave

Relación	Tipo	Descripción
ROL → USUARIO	1:N	Un rol puede tener muchos usuarios
USUARIO → CLIENTE	1:1	Solo clientes tienen perfil extendido
CLIENTE → SOLICITUD	1:N	Un cliente puede tener muchas solicitudes
SOLICITUD → DETALLE	1:N	Una solicitud tiene una o varias piezas
PIEZA → DETALLE	1:N	Una pieza puede estar en varias solicitudes
VEHICULO → INVENTARIO	1:N	Un vehículo puede tener muchas piezas extraídas
PIEZA → INVENTARIO	1:N	Una pieza puede extraerse de varios vehículos
PIEZA → CODIGO_QR	1:1	Cada pieza tiene su QR único
VEHICULO → CODIGO_QR	1:1	Cada vehículo tiene su QR único
PIEZA → MOVIMIENTO	1:N	Una pieza puede tener muchos movimientos de stock
USUARIO → NOTIFICACION	1:N	Un usuario puede recibir varias notificaciones

5. Tablas de la Base de Datos

1. ROLES

Campo	Tipo	Descripción
id_rol	INT PK AUTO	Identificador único
nombre	VARCHAR(50) UNIQUE	ADMIN, EMPLEADO, CLIENTE
descripcion	VARCHAR(255)	Descripción del rol

2. VEHICULOS

Campo	Tipo	Descripción
id_vehiculo	INT PK AUTO	Identificador único
matricula	VARCHAR(10) UNIQUE	Matrícula del vehículo
marca	VARCHAR(50)	Marca (Toyota, Ford...)
modelo	VARCHAR(50)	Modelo del vehículo
anio	INT	Año de fabricación
color	VARCHAR(30)	Color del vehículo
fecha_entrada	DATE	Fecha de entrada al desguace
estado	ENUM	completo / desguazando / desguazado
precio_compra	DECIMAL(10,2)	Precio pagado por el vehículo
kilometraje	INT	Kilómetros en el momento de la compra
ubicacion_gps	VARCHAR(50)	Referencia de ubicación en el patio
observaciones	TEXT	Notas internas

3. PIEZAS

Campo	Tipo	Descripción
id_pieza	INT PK AUTO	Identificador único
codigo_pieza	VARCHAR(20) UNIQUE	Código interno (ej: MOT-001)

Campo	Tipo	Descripción
nombre	VARCHAR(100)	Nombre de la pieza
categoria	ENUM	motor / carroceria / interior / electronica / ruedas / otros
precio_venta	DECIMAL(10,2)	Precio de venta al cliente
stock_disponible	INT	Unidades disponibles actualmente
stock_minimo	INT	Umbral que dispara la alerta de stock bajo
ubicacion_almacen	VARCHAR(50)	Referencia de ubicación en almacén
compatible_marcas	TEXT	Lista de vehículos compatibles
imagen	LONGTEXT	URL o base64 de la imagen
descripcion	TEXT	Descripción detallada

4. INVENTARIO_PIEZAS

Campo	Tipo	Descripción
id_vehiculo	INT FK	Vehículo del que se extrajo
id_pieza	INT FK	Pieza extraída
cantidad	INT	Cantidad extraída
estado_pieza	ENUM	nueva / usada / reparada
fecha_extraccion	DATE	Fecha de extracción
precio_unitario	DECIMAL(10,2)	Precio al que se extrajo
notas	VARCHAR(255)	Observaciones de la extracción

5. USUARIOS

Campo	Tipo	Descripción
id_usuario	INT PK AUTO	Identificador único
nombre	VARCHAR(100)	Nombre completo
email	VARCHAR(100) UNIQUE	Email y nombre de usuario
password_hash	VARCHAR(255)	Hash BCrypt de la contraseña

Campo	Tipo	Descripción
id_rol	INT FK	Rol asignado
activo	TINYINT(1)	1 = activo, 0 = desactivado
fecha_alta	DATETIME	Fecha de registro

6. CLIENTES

Campo	Tipo	Descripción
id_cliente	INT PK AUTO	Identificador único
id_usuario	INT FK UNIQUE	Usuario asociado
telefono	VARCHAR(20)	Teléfono de contacto
direccion	VARCHAR(255)	Dirección postal
nif	VARCHAR(15) UNIQUE	NIF/CIF del cliente

7. SOLICITUDES_PRESUPUESTO

Campo	Tipo	Descripción
id_solicitud	INT PK AUTO	Identificador único
id_cliente	INT FK	Cliente que realiza la solicitud
fecha_solicitud	DATETIME	Fecha y hora de creación
estado	ENUM	pendiente / en_revision / aprobada / rechazada
respuesta_admin	TEXT	Mensaje del administrador
precio_total	DECIMAL(10,2)	Precio acordado (se rellena al aprobar)
referencia_odoo	VARCHAR(50)	Número de pedido en Odoo (ej: SO/2026/0001)

8. DETALLE_SOLICITUD

Campo	Tipo	Descripción
id_solicitud	INT FK PK	Solicitud a la que pertenece
id_pieza	INT FK PK	Pieza solicitada

Campo	Tipo	Descripción
cantidad	INT	Número de unidades
notas	VARCHAR(255)	Observaciones del cliente

9. CODIGOS_QR

Campo	Tipo	Descripción
id_qr	INT PK AUTO	Identificador único
codigo_unico	VARCHAR(100) UNIQUE	Contenido del QR (ej: QR-PIE-00001)
tipo	ENUM	pieza / vehiculo
id_referencia	INT	ID de la pieza o vehículo referenciado
fecha_generacion	DATETIME	Fecha de generación

10. MOVIMIENTOS_STOCK

Campo	Tipo	Descripción
id_movimiento	INT PK AUTO	Identificador único
id_pieza	INT FK	Pieza afectada
tipo	ENUM	entrada / salida
cantidad	INT	Unidades del movimiento
id_usuario	INT FK	Empleado que realizó el movimiento
fecha	DATETIME	Fecha y hora del movimiento
notas	VARCHAR(255)	Motivo u observaciones

11. NOTIFICACIONES

Campo	Tipo	Descripción
id_notif	INT PK AUTO	Identificador único
id_usuario	INT FK	Usuario destinatario

Campo	Tipo	Descripción
tipo	ENUM	stock_bajo / solicitud_nueva / solicitud_actualizada / odoo_pedido / general
mensaje	TEXT	Contenido de la notificación
leida	TINYINT(1)	0 = no leída, 1 = leída
fecha_creacion	DATETIME	Fecha de creación

6. Roles de Usuario

El sistema tiene **tres roles** con permisos diferenciados en todos los endpoints:

ADMIN — Administrador

- Acceso completo a todas las funcionalidades
- Gestión de usuarios (crear, editar, desactivar)
- Aprobación o rechazo de solicitudes de presupuesto
- Al aprobar: integración automática con Odoo (crea pedido de venta)
- Gestión de vehículos y piezas (alta, baja, modificación)
- Visualización de todos los movimientos de stock y notificaciones
- Plataforma principal: **Desktop JavaFX**

EMPLEADO — Operario de almacén

- Consulta y actualización de stock de piezas
- Registro de movimientos de stock (entrada/salida)
- Consulta del listado de vehículos en patio
- Escaneo de QR para identificar piezas y vehículos
- Recepción de alertas de stock bajo
- Plataforma principal: **App Móvil Worker**

CLIENTE — Cliente registrado

- Registro y login en el Shop web
- Navegación por el catálogo de piezas
- Envío de solicitudes de presupuesto
- Consulta del estado de sus solicitudes

- Acceso al enlace de su pedido en Odoo cuando es aprobado
- Plataforma principal: **Web Shop**

Tabla de permisos por endpoint

Endpoint	ADMIN	EMPLEADO	CLIENTE
GET /api/piezas	✓	✓	✓
POST/PUT/DELETE /api/piezas	✓	✗	✗
GET /api/vehiculos	✓	✓	✗
POST/PUT/DELETE /api/vehiculos	✓	✗	✗
GET /api/stock/alertas	✓	✓	✗
POST /api/stock/movimiento	✓	✓	✗
POST /api/solicitudes	✓	✗	✓
GET /api/solicitudes (todas)	✓	✗	✗
GET /api/solicitudes (propias)	—	—	✓
PUT /api/solicitudes/{id}/aprobar	✓	✗	✗
GET /api/usuarios	✓	✗	✗
POST/PUT /api/usuarios	✓	✗	✗

7. Casos de Uso

Caso de uso 1 — Entrada de vehículo al desguace

Actor: Administrador (Desktop)

Flujo:

1. Admin inicia sesión en la app Desktop con `admin@autociclo.es`
2. Navega a "Vehículos" → "Nuevo vehículo"
3. Introduce matrícula, marca, modelo, año, precio de compra
4. El estado inicial es `completo`
5. El sistema genera automáticamente un código QR (QR-VEH-XXXXX) y lo registra en la BD
6. Se imprime la etiqueta QR y se coloca en el vehículo

Caso de uso 2 — Extracción y catalogación de pieza

Actor: Empleado (App Worker + Desktop)

Flujo:

1. Empleado abre la app Worker y escanea el QR del vehículo
2. Visualiza el estado del vehículo y sus piezas asociadas
3. Extrae la pieza y la lleva al almacén
4. En el Desktop, el admin registra la pieza en el inventario: vincula vehículo ↔ pieza, introduce estado (`usada`), fecha y precio
5. La pieza aparece en el catálogo web con stock disponible

Caso de uso 3 — Cliente busca y solicita una pieza

Actor: Cliente (Web Shop)

Flujo:

1. Cliente entra en `http://109.123.247.31:8090`
2. Busca por marca/modelo/categoría en el catálogo
3. Abre la ficha de la pieza: ve foto, precio, stock y compatibilidad
4. Hace clic en "Solicitar presupuesto"
5. Si no está registrado, se registra (`POST /api/auth/register`)
6. Rellena el formulario con las piezas deseadas y observaciones
7. La solicitud queda en estado `pendiente`
8. RabbitMQ publica un mensaje en `solicitudes.nueva` → el Desktop muestra badge de notificación

Caso de uso 4 — Aprobación de solicitud y generación de factura

Actor: Administrador (Desktop)

Flujo:

1. Admin ve el badge de notificación en el Desktop
2. Abre el módulo "Solicitudes" y revisa la solicitud pendiente
3. Comprueba disponibilidad de stock y precio
4. Hace clic en "Aprobar" e introduce el precio total y respuesta
5. Spring Boot llama a Odoo JSON-RPC: crea el cliente (si no existe) y genera el pedido de venta
6. Odoo devuelve la referencia `SO/2026/XXXX`
7. La solicitud pasa a estado `aprobada` con la referencia Odoo almacenada
8. RabbitMQ publica notificación al cliente
9. El cliente ve en "Mis Solicitudes" el estado actualizado con enlace al pedido en Odoo

Caso de uso 5 — Control de stock en almacén

Actor: Empleado (App Worker)

Flujo:

1. Empleado abre la app Worker → Dashboard
2. Ve las piezas con stock por debajo del mínimo (alertas en rojo/amarillo)
3. Localiza la pieza físicamente usando la ubicación en almacén
4. Escanea el QR de la pieza con la cámara del móvil
5. La app navega al detalle de la pieza
6. Empleado toca "Actualizar Stock" → selecciona `entrada` o `salida`, introduce cantidad y notas
7. Confirma el movimiento (Alert de confirmación)
8. Spring Boot registra el movimiento y actualiza `stock_disponible`
9. Si el nuevo stock sigue siendo bajo, publica mensaje en `stock.alerta`
10. El Dashboard se actualiza en el siguiente ciclo de polling (30 segundos)

Caso de uso 6 — Identificación rápida por QR

Actor: Empleado (App Worker)

Flujo:

1. Empleado encuentra una pieza sin identificar en el almacén
2. Abre la pestaña "Escanear QR" en la app
3. Apunta la cámara al código QR de la pieza
4. La app llama a `GET /api/codigos-qr/{codigo}`
5. Si es tipo `pieza`: navega directamente al detalle con toda su información
6. Si es tipo `vehiculo`: muestra un resumen del vehículo (marca, modelo, estado, matrícula)

8. Despliegue

Infraestructura del servidor

El sistema está desplegado en un **VPS Contabo** con Ubuntu 22.04 LTS:

```
IP: 109.123.247.31
```

```
Puerto 8080 → Spring Boot API (JAR ejecutado con systemd)
Puerto 8090 → Nginx (sirve el dist de React del Web Shop)
Puerto 8069 → Odoo 17 CE (proceso Python)
Puerto 5672 → RabbitMQ AMQP ──┐ Docker
Puerto 15672 → RabbitMQ UI ──┘ (contenedor: autociclo_rabbitmq)
Puerto 3306 → MySQL 8.0 (solo acceso local)
```

Docker — RabbitMQ

RabbitMQ se ejecuta en un contenedor Docker para aislar sus dependencias y facilitar su gestión:

```
# Arrancar RabbitMQ con Docker
docker run -d \
  --name autociclo_rabbitmq \
  --hostname rabbitmq \
  -p 5672:5672 \
  -p 15672:15672 \
  rabbitmq:3-management

# Verificar que está corriendo
docker ps | grep autociclo_rabbitmq
```

Las colas se crean automáticamente al arrancar la API gracias a la configuración en

`RabbitMQConfig.java` :

- `solicitudes.nueva` — notificaciones de nuevas solicitudes al Desktop
- `stock.alerta` — alertas de stock bajo al Worker

Spring Boot API (systemd)

```
# /etc/systemd/system/autociclo-api.service
[Unit]
Description=AutoCiclo API Spring Boot
After=network.target mysql.service

[Service]
ExecStart=/usr/bin/java -jar /opt/autociclo/autociclo-api.jar
User=autociclo
Restart=on-failure

[Install]
WantedBy=multi-user.target
```



```
# Comandos de gestión
systemctl start  autociclo-api
systemctl stop   autociclo-api
systemctl status autociclo-api
```

Nginx — Web Shop

```
# /etc/nginx/sites-available/autociclo-shop
server {
    listen 8090;
    root /var/www/autociclo-shop;
    index index.html;

    location / {
        try_files $uri $uri/ /index.html; # SPA fallback
    }
}
```

Proceso de despliegue de cambios

```
# 1. Compilar API en local
cd API/autociclo-api
mvn clean package -DskipTests
# Genera: target/autociclo-api-*.jar

# 2. Subir JAR y reiniciar servicio
scp target/autociclo-api-*.jar root@109.123.247.31:/opt/autociclo/autociclo-api.jar
ssh root@109.123.247.31 "systemctl restart autociclo-api"

# 3. Compilar Web Shop en local
cd Web/autociclo-shop
npm run build
# Genera: dist/

# 4. Subir dist a Nginx
scp -r dist/* root@109.123.247.31:/var/www/autociclo-shop/
```

Variables de entorno de la API (`application.properties`)

```
spring.datasource.url=jdbc:mysql://localhost:3306/autociclo_db
spring.datasource.username=autociclo_user
spring.datasource.password=***

jwt.secret=***
jwt.expiration=86400000

spring.rabbitmq.host=localhost
spring.rabbitmq.port=5672
spring.rabbitmq.username=guest
spring.rabbitmq.password=***

odoo.url=http://localhost:8069
odoo.db=autociclo
odoo.username=admin
odoo.password=***
```

9. API REST — Endpoints Principales

Base URL: <http://109.123.247.31:8080>

Autenticación: [Authorization: Bearer <token_jwt>](#)

Método	Endpoint	Rol	Descripción
POST	/api/auth/login	Público	Inicio de sesión → devuelve JWT
POST	/api/auth/register	Público	Registro de nuevo cliente
GET	/api/piezas	Público	Listado completo de piezas
GET	/api/piezas/buscar?q=	Público	Búsqueda por nombre/código
GET	/api/piezas/{id}	Público	Detalle de una pieza
POST	/api/piezas	ADMIN	Crear nueva pieza
PUT	/api/piezas/{id}	ADMIN	Actualizar pieza
GET	/api/vehiculos	EMPLEADO+	Listado de vehículos
POST	/api/vehiculos	ADMIN	Registrar vehículo
GET	/api/inventario/pieza/{id}	EMPLEADO+	Vehículo de origen de una pieza
GET	/api/stock/alertas	EMPLEADO+	Piezas con stock bajo
POST	/api/stock/movimiento	EMPLEADO+	Registrar entrada/salida de stock

Método	Endpoint	Rol	Descripción
POST	/api/solicitudes	CLIENTE	Crear solicitud de presupuesto
GET	/api/solicitudes	ADMIN	Ver todas las solicitudes
PUT	/api/solicitudes/{id}/aprobar	ADMIN	Aprobar + crear pedido en Odoo
GET	/api/codigos-qr/{codigo}	EMPLEADO+	Buscar pieza/vehículo por QR
GET	/api/notificaciones	Auth	Notificaciones del usuario

10. Usuarios de Prueba

Contraseña de todos los usuarios: Autociclo2026!

Email	Rol	Plataforma
admin@autociclo.es	ADMIN	Desktop
admin@autociclo.com	ADMIN	Desktop
pedro@autociclo.es	EMPLEADO	Worker (móvil)
operario@autociclo.com	EMPLEADO	Worker (móvil)
maria.garcia@email.com	CLIENTE	Web Shop
cliente@autociclo.com	CLIENTE	Web Shop

11. Manual de Usuario

11.1 Web Shop (Autociclo Shop)

URL: <http://109.123.247.31:8090>

Destinatario: Clientes del desguace

Navegación por el catálogo (sin registro)

1. Acceder a la URL del Web Shop — se muestra la pantalla de inicio con un buscador y las piezas más recientes.
2. Escribir en el buscador (ej. "motor", "faro", "rueda") — el catálogo filtra en tiempo real.

3. En la página </catalogo> se pueden aplicar filtros adicionales: categoría (motor, carrocería, interior...) y rango de precio.
4. Hacer clic en cualquier pieza para ver su ficha completa: nombre, código, stock disponible, precio, marcas compatibles y descripción.

Registro y solicitud de presupuesto

1. Clicar en **Registrarse** — rellenar nombre, email y contraseña.
2. Iniciar sesión con el email y contraseña registrados.
3. Desde la ficha de una pieza, clicar en **Solicitar presupuesto**.
4. Añadir las piezas deseadas y observaciones — confirmar el envío.
5. La solicitud queda en estado [pendiente](#).

Seguimiento de solicitudes

1. Ir a **Mis Solicitudes** en el menú superior.
2. Se muestran todas las solicitudes con su estado: [pendiente](#), [en_revision](#), [aprobada](#) o [rechazada](#).
3. Cuando el admin aprueba una solicitud, aparece el precio total y un enlace al pedido en Odoo ([S0/2026/XXXX](#)).

Usuarios de prueba para el Shop

Email	Contraseña
maria.garcia@email.com	Autociclo2026!
cliente@autociclo.com	Autociclo2026!
juan.martinez@email.com	Autociclo2026!

11.2 Desktop (Autociclo Desktop)

Plataforma: Windows / Linux con Java 21 instalado

Destinatario: Administradores del desguace

Inicio de sesión

1. Ejecutar la aplicación con [./gradlew run](#) (desarrollo) o el JAR distribuible.
2. Introducir email y contraseña de un usuario con rol ADMIN.
3. Si las credenciales son correctas, se carga el panel principal.

Gestión de vehículos

1. En el menú lateral, seleccionar **Vehículos**.

2. Se muestra el listado con matrícula, marca, modelo y estado.
3. Botón **Nuevo** para registrar un vehículo: rellenar matrícula, marca, modelo, año, precio de compra. El sistema genera un código QR automáticamente.
4. Doble clic sobre un vehículo para ver su ficha completa y las piezas asociadas.
5. Botón **Editar** para modificar datos (estado, ubicación, observaciones).

Gestión de piezas

1. En el menú lateral, seleccionar **Piezas**.
2. Listado con código, nombre, categoría, stock y precio.
3. Botón **Nueva pieza** para añadir al catálogo: código, nombre, categoría, precio, stock mínimo, ubicación en almacén.
4. Doble clic para ver el detalle y los vehículos de los que se extrajo.

Gestión de solicitudes y aprobación

1. Cuando llega una solicitud de un cliente, el badge de notificaciones en la barra lateral muestra un contador en rojo.
2. Seleccionar **Solicitudes** en el menú — aparece la solicitud en estado **pendiente**.
3. Revisar las piezas solicitadas y la disponibilidad de stock.
4. Clicar **Aprobar**: introducir el precio total y un mensaje al cliente.
5. El sistema llama automáticamente a Odoo JSON-RPC → crea el pedido de venta → devuelve la referencia (**SO/2026/XXXX**).
6. Para rechazar: clicar **Rechazar** e introducir el motivo.

Notificaciones RabbitMQ

- El badge en el menú lateral se actualiza en tiempo real cuando llega un nuevo mensaje a la cola **solicitudes.nueva**.
- Al marcar una notificación como leída, el contador se reduce.

Usuarios de prueba para Desktop

Email	Contraseña	Rol
admin@autociclo.es	Autociclo2026!	ADMIN
admin@autociclo.com	Autociclo2026!	ADMIN
supervisor@autociclo.es	Autociclo2026!	ADMIN

11.3 App Worker (Autociclo Worker)

Plataforma: Android (APK) o Expo Go para desarrollo

Destinatario: Empleados del almacén

Inicio de sesión

1. Abrir la app en el dispositivo.
2. Introducir email y contraseña de un usuario con rol EMPLEADO.
3. El token JWT se almacena de forma segura en `expo-secure-store` (cifrado).

Dashboard — Alertas de stock

- Al entrar, se muestran tres contadores: piezas **sin stock**, piezas **bajo mínimo** y **total de alertas**.
- Las tarjetas de alerta muestran el código de pieza, nombre, stock actual y mínimo.
- El Dashboard se actualiza automáticamente cada 30 segundos.
- Tocar una tarjeta navega directamente al detalle de la pieza.

Búsqueda de piezas

1. Ir a la pestaña **Buscar**.
2. Escribir nombre o código de pieza — resultados con debounce de 400ms.
3. Tocar un resultado para ver el detalle completo.

Escáner QR

1. Ir a la pestaña **Escanear QR**.
2. Apuntar la cámara al código QR de una pieza o vehículo.
3. La app consulta `GET /api/codigos-qr/{codigo}` :
 - Si es tipo `pieza` → navega al detalle de la pieza.
 - Si es tipo `vehículo` → muestra un resumen del vehículo.

Detalle de pieza y actualización de stock

1. En el detalle de una pieza se muestra: nombre, código, categoría, stock actual, stock mínimo, ubicación en almacén, precio y marcas compatibles.
2. Botones **+** y **-** para lanzar una actualización de stock.
3. El modal de confirmación solicita el tipo (entrada/salida), cantidad y nota opcional.
4. Al confirmar, se llama a `POST /api/stock/movimiento` y el stock se actualiza en tiempo real.

Listado de vehículos

1. Ir a la pestaña **Vehículos**.
2. Se muestran todos los vehículos con matrícula, marca, modelo, año y estado.
3. Campo de búsqueda para filtrar por marca o modelo.
4. Código de colores por estado: verde (completo), naranja (desguazando), gris (desguazado).

Usuarios de prueba para Worker

Email	Contraseña	Rol
pedro@autociclo.es	Autociclo2026!	EMPLEADO
operario@autociclo.com	Autociclo2026!	EMPLEADO
carlos@autociclo.es	Autociclo2026!	EMPLEADO

12. Estructura del Repositorio

```

Autociclo_TFG/
├── API/
│   ├── autociclo-api/           Spring Boot (Maven)
│   │   ├── src/main/java/com/autociclo/
│   │   │   ├── controllers/     Endpoints REST
│   │   │   ├── services/        Lógica de negocio
│   │   │   ├── models/          Entidades JPA
│   │   │   ├── dto/             Objetos de transferencia
│   │   │   ├── repositories/    Spring Data JPA
│   │   │   └── config/           Security, JWT, RabbitMQ
│   │   └── src/main/resources/
│   │       └── application.properties
│   └── Web/
│       ├── autociclo-shop/      React + Vite + TypeScript
│       │   ├── src/
│       │   │   ├── pages/        Páginas (Home, Catálogo, Ficha...)
│       │   │   ├── components/    Componentes reutilizables
│       │   │   ├── store/         Zustand (auth)
│       │   │   └── lib/           Axios, utilidades
│       └── Escritorio/
│           ├── AutoCiclo/        JavaFX (Gradle)
│           │   ├── app/src/main/
│           │   │   ├── java/      Controladores JavaFX
│           │   │   └── resources/fxml/ Ficheros FXML de interfaz
│           └── Autociclo_Worker/  React Native + Expo
│               ├── app/           Pantallas (Expo Router)
│               │   ├── login.tsx
│               │   ├── pieza/[id].tsx
│               │   └── (tabs)/     Dashboard, Buscar, QR, Vehículos
│               ├── lib/           API client (Axios), Auth (SecureStore)
│               └── store/         Zustand (auth)
├── BaseDatos/
│   ├── autociclo_db_v2.sql       Esquema completo + datos base
│   └── autociclo_db_demo.sql     Datos adicionales de demo (QR, solicitudes)
└── docs/
    └── DOCUMENTACION_PROYECTO.md (este documento)
  
```